Amortized Custom eFPGA Layout Search via Reinforcement Learning: Zero-Shot Generalization Across Circuit Benchmarks

Anonymous Author(s)

Abstract

Custom eFPGA fabrics embed reconfigurability in fixed silicon — enabling acceleration, post-fabrication updates, and IP security — but every search-based layout method to date restarts per netlist, repeating hundreds of VTR evaluations with nothing transferred from prior runs. We train a GNN-conditioned RL policy to place DSP/BRAM blocks and select fabric aspect ratio across 11 circuits; the same checkpoint then applies zero-shot to circuits never seen in training. Under a controlled comparison against a GA sharing the same oracle, metric, and baseline, zero-shot delivers an immediate positive result in one call while the GA requires 768 VTR evaluations and ~6.6 h — 50 generations spent stagnating after its best was already found. Jointly trained, the policy achieves 27.35% aggregate in-pool ADP reduction (three seeds); applied zero-shot to six held-out circuits — including fabrics up to 3.4× the training area — every result is positive, averaging 31.23%, with markedly lower seed variance than in-pool.

1 Introduction

Embedding a small reconfigurable fabric in an ASIC — an eFPGA — lets a fixed design adapt after fabrication: acceleration, protocol updates, IP security, and extended product life managed through the bitstream rather than a chip respin [20, 21]. A generic FPGA is over-provisioned for any single application — extra routing, DSP columns, and BRAMs the deployed netlist never fully uses. A custom eFPGA fabric, sized for one specific netlist, can recover much of that overhead. The product of routing area, critical-path delay, and dynamic power — **ADP** throughout — quantifies this overhead directly, and minimizing it benefits every eFPGA application [3, 20]. GOLDS [20] uses the same abbreviation for the two-term Area×Delay product; Section 5.3 recomputes on their exact metric to make the comparison direct.

Two prior papers show per-instance search substantially reduces eFPGA overhead: GOLDS [20] uses a GA with Area×Delay fitness and reports up to ~35% improvement on `diffeq1`; a follow-up applies NSGA-II and also co-evolves CLB LUT size and arithmetic-block sizing [3]. Both are *testbench-specific*: each run re-initializes from scratch, and nothing learned on one netlist transfers to the next — every new target pays the full search cost again.

We train a single GNN-conditioned RL policy to choose DSP/BRAM placement and fabric aspect ratio — the problem where per-instance GA search has been the only option. Unlike RL methods that assign blocks to an already-fabricated grid, our policy *constructs* the custom fabric layout itself. Trained jointly on 11 circuits and scored by the real VTR flow, the same checkpoint applies zero-shot to unseen circuits, including fabrics up to 3.4× the training pool area. Training runs once; each new netlist costs one rollout. We quantify the search-cost gap against a from-scratch GA under identical oracle, metric, and baseline: zero-shot gives an immediate positive result in one call; fine-tuned, the policy matches GA quality while the GA wastes most of its 768 evaluations stagnating past its early best (Section 5.2).

Concretely, this paper contributes:

- A GNN-conditioned RL policy for DSP/BRAM placement and aspect-ratio selection: one fixed-size checkpoint applies zero-shot to unseen circuits, evaluated against the true VTR flow rather than a proxy (Sections 3.3, 3.4).
- A controlled cost comparison on held-out `softmax`: zero-shot delivers an immediate positive result in one call; fine-tuned, the policy matches GA quality while the GA expends 768 evaluations (~6.6 h), stagnating 50 generations past its best at generation 46 (Section 5.2).
- 27.35% aggregate in-pool ADP reduction across three seeds (Section 5.1), and a zero-shot comparison against GOLDS on their exact metric: 37.4% vs. their ~35% (Section 5.3).
- A three-seed reliability analysis: in-pool fitting is far more seed-sensitive (two benchmarks spanning ~28 points) than zero-shot generalization (≤ 12 points across all six held-out benchmarks), reported as raw per-seed results (Sections 5.4, 5.6).

2 Background and Related Work

2.1 Custom eFPGA Layout Design

A generic island-style FPGA provisions DSP, BRAM, and routing for arbitrary workloads; any single application leaves much of this unused, paying area and delay for resources it never exercises. This motivates *custom* eFPGA layout: tailoring tile placement and fabric dimensions to a specific netlist for a compact, efficient fabric. Applications include IP security (function absent from fixed silicon until configured [6, 21]), domain-specific acceleration, and post-fabrication reconfigurability [20]. Subsequent work reduces overhead by shrinking under-utilized routing [15] or de-regularizing the fabric [8] — both motivating the non-island-style layouts this paper synthesizes. All rely on per-design search; none amortizes layout effort across netlists. GOLDS [20] introduced custom-tailored layout search: a GA over DSP/BRAM placement with Area×Delay fitness through the real VTR flow, reporting up to ~35% improvement on `diffeq1`. A follow-up [3] applies NSGA-II, also co-evolving CLB LUT size and DSP/BRAM block sizing, reporting up to ~44.6% improvement. Both start from a random population for every new design, transferring nothing from prior runs. We target the same problem GOLDS defines with the same VTR oracle, enabling a clean controlled comparison (Section 5.2); block-sizing co-evolution is a natural extension (Section 6).

2.2 Learned Policies for Physical Design

AlphaChip [11, 18] trains an RL agent to place ASIC macros using a learned proxy and reports cross-design transfer — the amortization argument we make here, at a much larger scale. Subsequent work explores visual representations [17], joint placement-and-routing [5], and transformer policies [16] — all supporting the claim: a trained

policy amortizes layout cost across designs as per-instance search cannot. Unlike AlphaChip, we evaluate every candidate against the true VTR flow, not a proxy — affordable at our scale, where a VTR run takes seconds to low minutes (Section 4.1).

Within the FPGA setting, RL has guided placement onto an *already-fabricated* grid — annealing perturbations in VPR [10] or a Gym-style VPR environment [4] — but always assigns blocks to a fixed grid, never shapes it. Transferable EDA policies exist for tool tuning [13] and global placement [12]; our policy instead synthesizes the custom fabric geometry.

2.3 VTR/VPR Background

We build on the open-source Verilog-to-Routing (VTR) [2, 9, 19]: Yosys synthesizes the design [7] and VPR packs, places, and routes against an architecture XML of tile types, positions, and connectivity. This XML — which GOLDS and NSGA-II generate by search and we generate by policy rollout — must represent arbitrary heterogeneous layouts, not just island-style grids, which is what makes this entire line of work possible.

3 Methodology

3.1 Problem Formulation

The reconfigurability here is not general-purpose: a fabric optimized for one netlist will not accommodate a circuit with substantially different DSP/BRAM requirements. This is intentional — generality is traded for a significantly smaller, more efficient implementation. Reconfigurability means updating the same primary function (parameter changes, post-fabrication fixes, a known set of operational modes), not repurposing the fabric for an unrelated workload.

Two roles are easily conflated: the *policy* designs the fabric; the *VTR toolchain* maps the circuit onto it. Given a netlist requiring n_{dsp} DSP and n_{bram} BRAM blocks, the policy emits $\pi = (\text{ar}, \{(x_i, y_i)\}_{i=1}^{n_{\text{dsp}}+n_{\text{bram}}})$: a fabric aspect ratio $\text{ar} \in \mathcal{R}$ and a tile coordinate for every hard block. VTR then synthesizes, packs, places, routes, and auto-sizes the array to the smallest grid of the chosen aspect ratio that holds the design. The policy never places a CLB or fixes grid dimensions — it shapes the fabric and positions hard blocks; VTR reports back the realized cost. We take that cost to be the three-term ADP (Section 1):

$$\text{ADP}(\pi) = \text{Area}(\pi) \times \text{Delay}(\pi) \times \text{Power}(\pi), \tag{1}$$

where each term is from the VTR flow (Section 4). We use VPR's *routing area* — the physical tile-grid footprint including routing channels — not CLB count: the policy controls fabric geometry, which changes routing area; logical CLB area is layout-independent. Routing area is the metric that responds to policy decisions and reflects actual on-chip die area. We report results as percentage ADP reduction relative to a traditional island-style baseline at matched logic capacity.

3.2 RL Formulation

We pose placement as a finite-horizon sequential decision process and train with Maskable PPO [14] (`MaskablePPO`), which restricts the action space to legal tile coordinates at each step.

Episode structure. Step 0 selects the fabric aspect ratio from $\mathcal{R}$; each subsequent step positions one hard block (BRAMs then DSPs, both ordered by descending shared-net connectivity). Once all blocks are placed, the environment bakes the layout into an architecture XML via Jinja2 and invokes VTR to obtain $\text{Area}(\pi)$, $\text{Delay}(\pi)$, $\text{Power}(\pi)$; intermediate steps receive zero reward.

Action space. The action space is $\text{Discrete}(W_{\max} \times H_{\max})$, fixed across all benchmarks (Section 3.4). Step-0 actions select from $\mathcal{R}$; later steps decode an index to (x, y) via a fixed stride on $H_{\max}$. Action masking restricts the policy to legal, non-overlapping, in-bounds coordinates for the active benchmark.

Reward. The terminal reward is a weighted sum of negative log-ratios against the traditional baseline for the active benchmark:

$$r = -\left(w_a \log \frac{\text{Area}}{\text{Area}_0} + w_d \log \frac{\text{Delay}}{\text{Delay}_0} + w_p \log \frac{\text{Power}}{\text{Power}_0} \right), \tag{2}$$

where $\text{Area}_0, \text{Delay}_0, \text{Power}_0$ are the traditional-baseline metrics and w_a, w_d, w_p are configurable (Section 3.5). With $w_a = w_d = w_p = 1$, $r = \log(\text{ADP}_0/\text{ADP})$, making reward and headline metric directly comparable. An invalid choice (degenerate aspect ratio, out-of-bounds or overlapping placement, or a VTR failure) terminates early with penalty -10.

3.3 GCN Feature Extractor

To generalize across benchmarks with varying netlist sizes, hard-block counts, and grid dimensions, we encode each netlist as a graph, giving the policy the inductive bias for cross-topology transfer that graph learning provides elsewhere in EDA [22]. We reduce each netlist to a compact fixed-schema graph: one node per DSP/BRAM instance plus one aggregate "fabric" node for the remaining standard-cell logic. Each node carries `[is_dsp, is_bram, is_fabric, net_count_normalized]`. Hard-block–hard-block edges are weighted by shared-net count; hard-block–fabric edges by summed shared-net count across touched standard cells. Both edge types are log-normalized into $[0, 1]$ separately — $w_{\text{norm}} = \log(1 + w)/\log(1 + c)$ with per-kind ceiling c — so neither saturates or vanishes.

The graph is padded to a fixed (MAX_NODES, MAX_EDGES) (Section 3.4) and fed to a shared GNN: two `GCNConv` layers (width 64) produce per-node embeddings; global mean-pooling gives a whole-netlist embedding, and the current-node embedding gives a per-step embedding. These are concatenated with a CNN over the 4-channel occupancy grid and the benchmark's normalized (width, height), then projected to a 128-d vector for the PPO heads. The extractor trains end-to-end under the PPO loss, with no separate GNN pretraining.

3.4 Universe Superset and Action Masking

Policy dimensions are fixed at construction, yet we want one checkpoint loadable against benchmarks never trained on. We therefore size all dimension-dependent quantities — $(W_{\max}, H_{\max})$, MAX_NODES, MAX_EDGES — from a deliberate superset *universe* that includes benchmarks reserved for zero-shot evaluation. Per-episode action masking and the `valid_wh` observation let the fixed-size network operate on each benchmark's smaller real footprint.

3.5 Training Setup

Training samples uniformly across the 11-benchmark pool episode-by-episode, updating the policy jointly on all 11 circuits. We use `MaskablePPO` with 24 parallel environments, 48 rollout steps per

update, batch size 144, 15 epochs per update, entropy coefficient 0.01, and reward weights $w_a = w_d = w_p = 1$ (Eq. 2), giving reward = $\log(\text{ADP}_0/\text{ADP})$. Each run trains for 13,000 episodes with a 300s VTR timeout per episode to prevent a degenerate rollout from stalling the 23 parallel workers. We report three independent seeds (7, 42, 123) throughout Section 5 to characterize reliability rather than present a single run as ground truth.

4 Experimental Setup

4.1 Evaluation Oracle

Every candidate placement is scored by the true toolchain, not a proxy. A placement is rendered into a VTR architecture XML and VPR constraints file via Jinja2 templates, then run through the full VTR flow — synthesis, packing, placement, routing, and power estimation against a 45nm PTM file — and routing area, critical-path delay, and dynamic power are extracted from VPR reports. All evaluations use VTR 9.0.0 (git `22a09d39`, GCC 13.3.0). Evaluated pairs are cached; the synthesis-to-routing flow dominates cost (seconds to low minutes per call).

4.2 Baseline

The baseline is VTR's default island-style template (`k6_frac_N10_mem32K_40nm`: 6-input LUTs, cluster size 10, 32K-bit BRAMs, 40nm) at matched logic capacity — the same template GOLDS [20] uses, enabling direct percentage comparison (Section 5.3).

4.3 Benchmarks

Figure 1 lists all 17 benchmarks with grid size and hard-block counts: 11 for training and 6 held-out for zero-shot evaluation only (Section 5.4). Training circuits come from the VTR suite [19]; four held-out benchmarks (`softmax`, `reduction_layer`, `arm_core`, `mkDelayWorker32B`) are larger circuits from Koios [1] and the VTR suite, probing fabric sizes well beyond the training pool. Two small probes — `cipher` and `macbuf` (an internally built multiply-accumulate buffer) — cover the small-fabric end, which standard suites underrepresent. The GA head-to-head (Section 5.2) uses `softmax`, a Koios benchmark held out of RL training.

4.4 Baselines Compared

We compare against: (1) the traditional island-style eFPGA baseline (Section 4.2); (2) a genetic algorithm we implemented under the same evaluation oracle and ADP metric (Section 5.2); and (3) published results from GOLDS [20] and the NSGA-II follow-up [3], subject to the metric caveat in Section 5.3.

5 Results

5.1 In-Pool Performance

Figure 1 (upper group) shows per-benchmark in-pool ADP reduction across three seeds (7, 42, 123); we report raw outcomes rather than mean ± SD, which three runs cannot justify (Section 5.6). Each seed's pooled aggregate — one minus the ratio of summed ADP to summed baseline ADP, *not* a simple mean of per-benchmark percentages — is 35.73%, 23.11%, and 23.97% (mean 27.35%). `raygentop` is consistently negative ($[-2.2, -1.7]$%). Its 6-DSP/1-BRAM profile is unique in the pool (other DSP-heavy benchmarks have no BRAM;

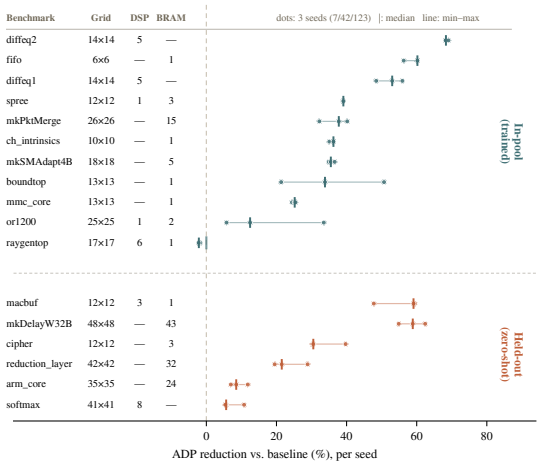

Figure 1: Per-seed ADP reduction for all 17 benchmarks; row labels encode grid size and hard-block counts (D=DSP, B=BRAM) in place of a separate table. Dots are three seeds (7/42/123), tick is median, line spans min–max. `or1200` and `boundtop` span ∼28–29 points across seeds; every other benchmark stays within ≤ 12. The held-out group is more seed-stable than in-pool.

BRAM-having ones have ≤ 1 DSP), and despite a ≈9.1% episode share the policy finds no improving placement — plausibly a representational gap, and our most immediate diagnostic target. `or1200` and `boundtop` account for nearly all seed-to-seed instability: their ranges (5.8–33.5% and 21.3–50.7%) are 3–4× wider than any other benchmark (≤ 8 points); see Section 5.6.

5.2 GA Head-to-Head Comparisons

For a controlled cost comparison without the metric caveat of Section 5.3, we implement a GA under the same oracle, ADP metric, and baseline on `softmax`: roulette-wheel selection, single-point crossover, per-gene mutation over hard-block coordinates and aspect ratio, population 8, capped at 500 generations with patience 50.

Table 1 reports results. The GA's best — **14.62%** ADP reduction — appeared at generation 46 after 368 evaluations and ∼2.75 h. It then stagnated for 50 generations (400 more evaluations, ∼3.8 h), for a total of 768 calls and ∼6.6 h. Zero-shot, the policy achieves **5.39%** in a *single* call (∼134 s); fine-tuned, it matches the GA at ∼episode 552 (∼4 h) and continues to 14.82% at episode 1,128 (∼8 h).

The timing exposes a structural difference: the GA's best appeared at generation 46 and never improved; the remaining 50 generations were spent on a stagnated population. Per-generation averages confirm it — the typical individual is worse than baseline in *every* generation; the GA carries forward a lucky draw but never updates its sampling distribution. Gradient updates do exactly this: the fraction of RL episodes beating baseline rises from ∼10% to ∼75% by the end, which is why the policy continues past the GA's stall point. The GA's cost is re-incurred per design; the policy's is fixed, so the amortization advantage grows with the number of designs.

Table 1: GA vs. RL on `softmax` (held out of RL training; baseline ADP = 3.58×10^6). GA: pop. 8, ≤500 gens, patience 50; best at gen. 46, stagnated the final 50 gens. RL zero-shot: single deterministic rollout (seed 42). RL fine-tuned: 11-benchmark policy continued on `softmax`. Same 24-core machine throughout.

Method	Reduction	VTR calls	Wall-clock
GA	14.62%	768	∼6.6 h
RL zero-shot	5.39%	**1**	**∼2 min**
RL fine-tuned	**14.82%**	1,128	∼8 h

Figure 2 shows the physical saving on `softmax`. The traditional fabric provisions BRAM columns the netlist never uses; the policy omits them and selects an aspect ratio under which VTR auto-sizes the logic to a 22% smaller array — the dominant ADP term. The policy never moves a CLB; it removes idle columns and reshapes the fabric, and the tighter routing is VTR's.

5.3 Head-to-Head Against GOLDS

As a literature-facing comparison, we recompute a zero-shot result — `diffeq2`-trained policy on `diffeq1` — using GOLDS' exact two-term Area×Delay metric and their baseline. Two differences apply: (1) we use the two-term product, not our three-term ADP; (2) GOLDS' area term is a logical utilization measure while ours is VPR's routing area (Section 3.1); we recompute on their metric and baseline to match as closely as possible. This yields **37.4%** versus GOLDS' ∼**35%** — a real margin, achieved with zero benchmark-specific training. Nowhere else do we compare two-term against three-term numbers.

5.4 Zero-Shot Generalization

Figure 1 (lower group) reports ADP reduction on six held-out benchmarks across the same three seeds, with zero benchmark-specific training. Two (`softmax`, `reduction_layer`) were in the sizing universe but never sampled; the other four were outside the universe entirely, a strictly harder test. Four of the six are larger than anything in the training pool (up to 2,304 tiles vs. 676), making this size extrapolation, not interpolation. Every benchmark is positive under every seed — no sign flips — and every per-seed range is narrow (≤ 12 points), far from the `or1200`/`boundtop` spread. The overall average is **31.23%**.

All numbers use greedy (deterministic) decoding. As a diagnostic, stochastic rollouts on one seed yielded negative average ADP on 4 of 6 benchmarks: the greedy action occupies a high-quality mode, but most probability mass does not. Zero-shot results are top-choice performance per seed, not a sampling expectation.

5.5 Fine-Tuning Efficiency

We compare fine-tuning the multi-benchmark policy against training from scratch on `diffeq1`: 6,000 episodes each, identical hyperparameters, separate caches. The fine-tuned run dominates throughout (0.785 vs. 0.736 final-window reward) and plateaus by ≈episode 3,000; scratch never plateaus. The mechanism is entropy collapse: fine-tuned entropy falls 2.00→0.79 (scratch: 4.63→2.42), so only 31.5% of its episodes are novel VTR evaluations vs. 94.4% for scratch — a sample-efficiency win that also caps exploration beyond the pretrained prior (Section 5.6). This is continued learning, not cross-benchmark transfer (Section 5.4).

5.6 Seed Variance: In-Pool vs. Zero-Shot

Across three seeds, a clear asymmetry emerges: *in-pool* performance is considerably more seed-sensitive than *zero-shot*. Nine of eleven training benchmarks show ≤ 8 points of spread; `or1200` and `boundtop` span ∼28–29 points — 3–4× any other — accounting for almost the entire pooled aggregate spread (35.73%/23.11%/23.97%). On the six held-out benchmarks, every range stays within ∼12 points (Figure 1).

We propose a mechanism: in-pool performance depends on which benchmarks a seed converges on slowly within the shared budget, while zero-shot reflects a more general property of the learned representation (Section 3.3). Entropy collapse (Section 5.5) compounds this: early convergence on most benchmarks leaves fewer episodes for slow-to-converge ones like `or1200`. No seed leads on both high-variance benchmarks and elsewhere (seed 42 leads on `or1200`/`boundtop`; seeds 7 and 123 lead on others), pointing to per-benchmark convergence variance, not a globally better seed. At $n = 3$ this is a characterization; more seeds would establish whether the asymmetry is stable.

6 Conclusion

One policy trained across 11 circuits answers the cost question directly: under controlled comparison, the GA stagnates after generation 46, spending 768 evaluations and ∼6.6 h past its early best. The fine-tuned policy matches that quality and keeps improving; zero-shot, one call delivers an immediate positive result (Section 5.2). The same policy generalizes zero-shot to six held-out benchmarks — four larger than anything in training — and exceeds GOLDS' result on their own metric (Sections 5.3, 5.4).

Two findings qualify these results: in-pool fitting is far more seed-sensitive than zero-shot transfer (23.11–35.73% across seeds, driven by `or1200` and `boundtop`), and deterministic zero-shot numbers diverge sharply from stochastic sampling (negative for 4 of 6 benchmarks). A single seed or rollout is not a stable characterization (Section 5.6).

Three gaps remain: results rest on three seeds with no systematic stochastic-rollout evaluation; the action space covers placement and aspect ratio only, where co-evolving CLB LUT size and block sizing [3] is a natural extension; and broadening to the full GOLDS/NSGA-II benchmark set would enable direct comparison without the metric conversion of Section 5.3.

References

[1] Aman Arora, Andrew Boutros, Daniel Rauch, Aishwarya Rajen, Aatman Borda, Seyed A. Damghani, Samidh Mehta, Sangram Kate, Pragnesh Patel, Kenneth B. Kent, Vaughn Betz, and Lizy K. John. 2021. Koios: A Deep Learning Benchmark Suite for FPGA Architecture and CAD Research. In *31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE. doi:10.1109/FPL53798.2021.00010

[2] Vaughn Betz and Jonathan Rose. 1997. VPR: A New Packing, Placement and Routing Tool for FPGA Research. In *7th International Workshop on Field-Programmable Logic and Applications (FPL)*. Springer-Verlag, 213–222. doi:10.1007/3-540-63465-7_226

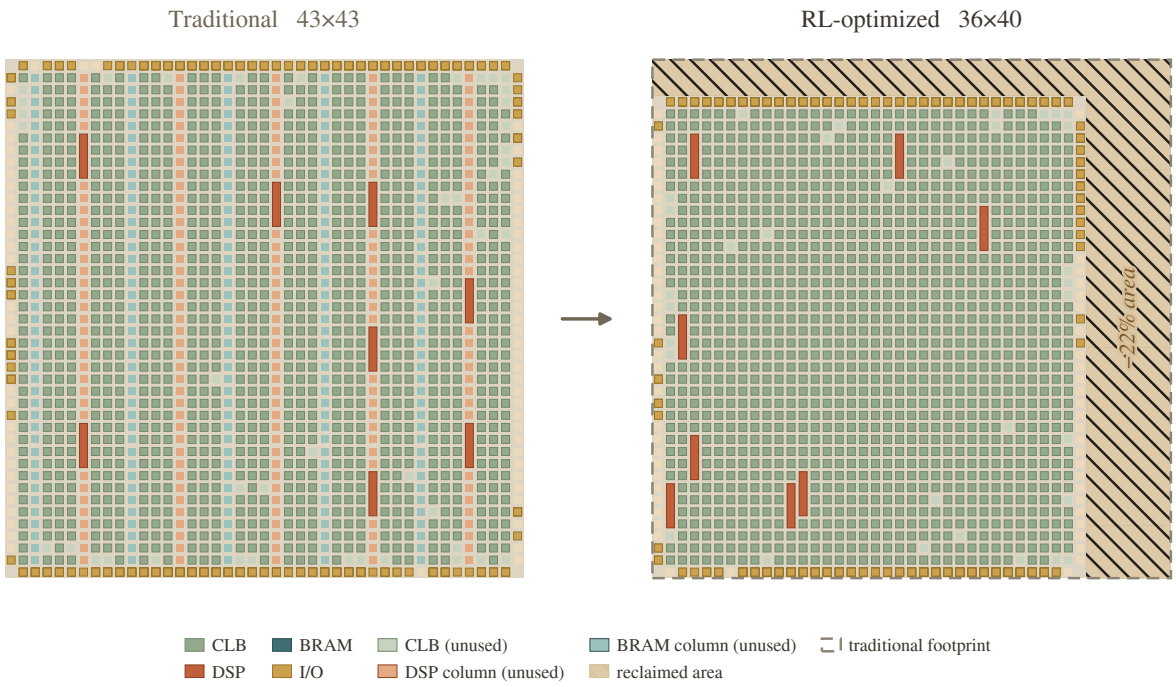

Figure 2: What the policy recovers, physically (`softmax`, the GA comparison benchmark). Both fabrics are drawn at the *same tile scale*. The traditional island-style baseline (left) is a 43×43 heterogeneous fabric that provisions BRAM columns (plum) the netlist never uses, with DSP blocks (terracotta) spread across regular DSP columns. The RL-optimized fabric (right) tailors the architecture to the netlist: it drops the unused BRAM columns entirely and repositions the DSPs, under an aspect ratio that lets VTR auto-size the same logic down to 36×40. The dashed outline is the traditional footprint and the hatched strip is the reclaimed area — a 22% smaller fabric, the dominant term in the 15% ADP reduction. Gaps between tiles are routing channels.

[3] D. V. Bhargav, G. Pradyumna, and Madhav Rao. 2025. Meta-Heuristic Optimization of Custom Heterogeneous Blocks Defined eFPGA Design. In *38th International Conference on VLSI Design (VLSID)*. IEEE, 326–331. doi:10.1109/VLSID64188.2025.00069

[4] Ruichen Chen, Shengyao Lu, Mohamed A. Elgammal, Peter Chun, Vaughn Betz, and Di Niu. 2023. VPR-Gym: A Platform for Exploring AI Techniques in FPGA Placement Optimization. In *33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 72–78. doi:10.1109/FPL60245.2023.00018

[5] Ruoyu Cheng and Junchi Yan. 2021. On Joint Learning for Solving Placement and Routing in Chip Design. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 16508–16519.

[6] Luca Collini, Jitendra Bhandari, Chiara Muscari Tomajoli, Abdul Khader Thalakkattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2025. ARIANNA: An Automatic Design Flow for Fabric Customization and eFPGA Redaction. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* (2025). volume/issue/pages unverified at press time. doi:10.1145/3737287

[7] Seyed Alireza Damghani and Kenneth B. Kent. 2022. Yosys+Odin-II: The Odin-II Partial Mapper with Yosys Coarse-grained Netlists in VTR. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 157. doi:10.1145/3490422.3502344

[8] Voktho Das, Kimia Zamiri Azar, and Hadi M. Kamali. 2026. NuRedact: Non-Uniform eFPGA Architecture for Low-Overhead and Secure IP Redaction. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE.

[9] Mohamed A. Elgammal, Amin Mohaghegh, Soheil G. Shahrouz, Fatemehsadat Mahmoudi, Fahrican Kosar, Kimia Talaei, Joshua Fife, Daniel Khadivi, Kevin E. Murray, Andrew Boutros, Kenneth B. Kent, Jeffrey B. Goeders, and Vaughn Betz. 2025. VTR 9: Open-Source CAD for Fabric and Beyond FPGA Architecture Exploration. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 18, 3 (2025), 39:1–39:53. doi:10.1145/3734798

[10] Mohamed A. Elgammal, Kevin E. Murray, and Vaughn Betz. 2022. RLPlace: Using Reinforcement Learning and Smart Perturbations to Optimize FPGA Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 41, 8 (2022), 2532–2545. doi:10.1109/TCAD.2021.3109863

[11] Anna Goldie, Azalia Mirhoseini, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2024. Addendum: A graph placement methodology for fast chip design. *Nature* 634, 8034 (2024), E10–E11. doi:10.1038/s41586-024-08032-5

[12] Yunbo Hou, Haoran Ye, Shuwen Yang, Yingxue Zhang, Siyuan Xu, and Guojie Song. 2025. TransPlace: Transferable Circuit Global Placement via Graph Neural Network. *arXiv preprint arXiv:2501.05667* (2025).

[13] Hao-Hsiang Hsiao, Yi-Chen Lu, Pruek Vanna-Iampikul, and Sung Kyu Lim. 2024. FastTuner: Transferable Physical Design Parameter Optimization using Fast Reinforcement Learning. In *International Symposium on Physical Design (ISPD)*. ACM.

[14] Shengyi Huang and Santiago Ontañón. 2022. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. In *Proceedings of the 35th International FLAIRS Conference*, Vol. 35. doi:10.32473/flairs.v35i.130584

[15] Hadi M. Kamali, Kimia Z. Azar, Farimah Farahmandi, and Mark Tehranipoor. 2023. SheLL: Shrinking eFPGA Fabrics for Logic Locking. In *Design, Automation & Test in Europe Conference (DATE)*. IEEE, 1–6.

[16] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. 2023. ChiPFormer: Transferable Chip Placement via Offline Decision Transformer. In *Proceedings of the 40th International Conference on Machine Learning (ICML) (PMLR, Vol. 202)*. 18346–18364.

[17] Yao Lai, Yao Mu, and Ping Luo. 2022. MaskPlace: Fast Chip Placement via Reinforced Visual Representation Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 24019–24030.

[18] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nova, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2021.

A graph placement methodology for fast chip design. *Nature* 594, 7862 (2021), 207–212. doi:10.1038/s41586-021-03544-w

[19] Kevin E. Murray, Oleg Petelin, Sheng Zhong, Jia Min Wang, Mohamed Eldafrawy, Jean-Philippe Legault, Eugene Sha, Aaron G. Graham, Jean Wu, Matthew J. P. Walker, Hanqing Zeng, Panagiotis Patros, Jason Luu, Kenneth B. Kent, and Vaughn Betz. 2020. VTR 8: High-performance CAD and Customizable FPGA Architecture Modelling. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 13, 2 (2020), 9:1–9:55. doi:10.1145/3388617

[20] Pratyush Nandi, Anubhav Mishra, and Madhav Rao. 2024. GOLDS: Genetic Algorithm-based Optimization of Custom FPGA Architecture Layout Design for Secure Silicon. In *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI)*. ACM, 92–97. doi:10.1145/3649476.3658743

[21] Chiara Muscari Tomajoli, Luca Collini, Jitendra Bhandari, Abdul Khader Thalakkattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2022. ALICE: An Automatic Design Flow for eFPGA Redaction. In *59th ACM/IEEE Design Automation Conference (DAC)*. 781–786. doi:10.1145/3489517.3530543

[22] Nan Wu et al. 2023. GAMORA: Graph Learning based Symbolic Reasoning for Large-Scale Boolean Networks. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.